

I. Introduction:

In this assignment, we aimed to explore the functionality and performance of various RNN-based architectures. Specifically, we focused on four main architectures: Standard LSTM, Standard GRU, Multiplicative LSTM, and Multiplicative GRU. These architectures were applied to a COPY task designed to evaluate their ability to retain and reproduce sequences after a specified delay.

The COPY task involves the following steps:

- Generate a sequence of random symbols with length T
- T (e.g., from a small vocabulary of 8–10 tokens).
- Present the sequence to the model, followed by a delimiter (or blank token) for several steps.
- After the delimiter, the model is tasked with outputting (copying) the original sequence of length T

The task was tested on sequence lengths

$$T \in \{100, 200, 500, 1000\}$$

As T increases, the challenge for the model grows, requiring it to store information in memory for longer durations. This setup highlights differences in the gating mechanisms of the various RNN architectures and their ability to handle long-term dependencies effectively.

II. Methods & Results:

1. Data Generation

Firstly, we generated the data for the copy task using the following specifications:

a) **Vocabulary:** The vocabulary was defined as follows:

```
# Define the vocab
vocabulary = ['N', 'L', 'P', 'I', 'S', 'F', 'U', 'Z']
delimiter = '##'
vocab_size = len(vocabulary) + len(delimiter) # 10 in total
```

c) **Encoding:** We converted each sequence to a character level one-hot encoded vector.

b) **Samples per length:** We generated 100 samples for each of the lengths

$$T \in \{100, 200, 500, 1000\}$$

c) **Train, validation and Test sets:** As specified in the requirements, the training and validation was only performed on the sequences of lengths 100, 250, and 500. All the sequences of length 1000 were only in testing but not in training.

Our train and validation split was 80-20, which was performed using the random_split method from pytorch to **prevent any data leaks**.

```
# Split the train_val dataset
train_dataset, val_dataset = torch.utils.data.random_split(
    train_val_dataset, [train_size, val_size], generator=cuda_generator
)
```

All the data was converted to tensors to allow easy training.

So, following were the final sizes of each set:

Train set: 240 samples, $T \in \{100, 200, 500\}$

Validation set: 60 Samples, $T \in \{100, 200, 500\}$

Test Set: 100 Samples, $T \in \{100, 200, 500\}$

Padding was performed to account for varying sequence lengths.

2. Models

We defined 4 main architectures based on the specified equations

2.1 Standard LSTM

$$\begin{aligned} \mathbf{i}_t &= \sigma(\mathbf{W}^{(i)}\mathbf{x}_t + \mathbf{U}^{(i)}\mathbf{h}_{t-1} + \mathbf{b}^{(i)}), \\ \mathbf{f}_t &= \sigma(\mathbf{W}^{(f)}\mathbf{x}_t + \mathbf{U}^{(f)}\mathbf{h}_{t-1} + \mathbf{b}^{(f)}), \\ \mathbf{o}_t &= \sigma(\mathbf{W}^{(o)}\mathbf{x}_t + \mathbf{U}^{(o)}\mathbf{h}_{t-1} + \mathbf{b}^{(o)}), \\ \tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}^{(c)}\mathbf{x}_t + \mathbf{U}^{(c)}\mathbf{h}_{t-1} + \mathbf{b}^{(c)}), \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \end{aligned}$$

2.2 Multiplicative LSTM

Introduce a learnable *memory matrix* $\mathbf{M}_t$ or vector $\mathbf{m}_t$:

$$\mathbf{m}_t = \mathbf{W}_m \mathbf{x}_t + \mathbf{U}_m \mathbf{h}_{t-1} + \mathbf{b}_m, \quad \mathbf{M}_t = \text{diag}(\mathbf{m}_t),$$

and define

$$\tilde{\mathbf{x}}_t = \mathbf{M}_t \mathbf{x}_t \text{ or } \tilde{\mathbf{x}}_t = \mathbf{m}_t \odot \mathbf{x}_t.$$

Then substitute $\tilde{\mathbf{x}}_t$ in place of $\mathbf{x}_t$ in the LSTM equations:

$$\begin{aligned} \mathbf{i}_t &= \sigma(\mathbf{W}^{(i)}\tilde{\mathbf{x}}_t + \mathbf{U}^{(i)}\mathbf{h}_{t-1} + \mathbf{b}^{(i)}), \\ \mathbf{f}_t &= \sigma(\mathbf{W}^{(f)}\tilde{\mathbf{x}}_t + \mathbf{U}^{(f)}\mathbf{h}_{t-1} + \mathbf{b}^{(f)}), \\ \mathbf{o}_t &= \sigma(\mathbf{W}^{(o)}\tilde{\mathbf{x}}_t + \mathbf{U}^{(o)}\mathbf{h}_{t-1} + \mathbf{b}^{(o)}), \\ \tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}^{(c)}\tilde{\mathbf{x}}_t + \mathbf{U}^{(c)}\mathbf{h}_{t-1} + \mathbf{b}^{(c)}), \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \end{aligned}$$

2.3 Standard GRU

$$\begin{aligned} \mathbf{z}_t &= \sigma(\mathbf{W}^{(z)}\mathbf{x}_t + \mathbf{U}^{(z)}\mathbf{h}_{t-1} + \mathbf{b}^{(z)}), \\ \mathbf{r}_t &= \sigma(\mathbf{W}^{(r)}\mathbf{x}_t + \mathbf{U}^{(r)}\mathbf{h}_{t-1} + \mathbf{b}^{(r)}), \\ \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}^{(h)}\mathbf{x}_t + \mathbf{U}^{(h)}(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}^{(h)}), \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \end{aligned}$$

2.4 Multiplicative GRU

Again, define

$$\mathbf{m}_t = \mathbf{W}_m \mathbf{x}_t + \mathbf{U}_m \mathbf{h}_{t-1} + \mathbf{b}_m, \quad \tilde{\mathbf{x}}_t = \mathbf{m}_t \odot \mathbf{x}_t \quad \text{or} \quad \tilde{\mathbf{x}}_t = \mathbf{M}_t \mathbf{x}_t.$$

Then substitute $\tilde{\mathbf{x}}_t$:

$$\begin{aligned} \mathbf{z}_t &= \sigma(\mathbf{W}^{(z)} \tilde{\mathbf{x}}_t + \mathbf{U}^{(z)} \mathbf{h}_{t-1} + \mathbf{b}^{(z)}), \\ \mathbf{r}_t &= \sigma(\mathbf{W}^{(r)} \tilde{\mathbf{x}}_t + \mathbf{U}^{(r)} \mathbf{h}_{t-1} + \mathbf{b}^{(r)}), \\ \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}^{(h)} \tilde{\mathbf{x}}_t + \mathbf{U}^{(h)} (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}^{(h)}), \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \end{aligned}$$

Refer the jupyter notebook for the model implementations using pytorch.

3. Hyperparameters and Significance Testing:

We reported the loss and accuracy for each of the following hyperparameters:

Learning_rate = {0.1, 0.001, 0.0001}

Hidden_sizes = {64, 128}

Epochs = {10, 20}

We performed a grid search on the above mentioned hyper parameters.

Furthermore, we ran the whole experiment with 3 different random seeds: **000, 12, 88**

Following are our results for each of the above mentioned configs on the test set, for Standard LSTM with random seed = 000, best results highlighted in yellow

Learning rate	Hiddensize	Epochs	Simple Lstm Accuracy	Standard LSTM Loss
0.001	64	10	0.0735	0.3750
0.001	64	20	0.0577	0.5859

0.001	128	10	0.0631	0.4531
0.001	128	20	0.0300	0.9062
0.0001	64	10	0.0930	0.2422
0.0001	64	20	0.0881	0.2109
0.0001	128	10	0.0880	0.1094
0.0001	128	20	0.0850	0.2500

Following are our results for each of the above mentioned configs on the test set, for Multiplicative LSTM with random seed = 000, best results highlighted in yellow

Learning rate	Hiddensize	Epochs	Simple Lstm Accuracy	Standard LSTM Loss
0.001	64	10	0.0833	0.2891
0.001	64	20	0.0377	0.8203
0.001	128	10	0.0572	0.5703

0.001	128	20	0.0112	0.8100
0.0001	64	10	0.0960	0.1172
0.0001	64	20	0.0925	0.2109
0.0001	128	10	0.0921	0.1094
0.0001	128	20	0.0910	0.1094

Following are our results for each of the above mentioned configs on the test set, for Standard GRU with random seed = 000:

Learning rate	Hiddensize	Epochs	Simple Lstm Accuracy	Standard LSTM Loss
0.001	64	10	0.0692	0.7500
0.001	64	20	0.0498	0.8281
0.001	128	10	0.0473	0.7969
0.001	128	20	0.0143	0.8130

0.0001	64	10	0.0996	0.0703
0.0001	64	20	0.0896	0.1953
0.0001	128	10	0.0865	0.2656
0.0001	128	20	0.0800	0.5156

Following are our results for each of the above mentioned configs on the test set, for Multiplicative GRU with random seed = 000:

Learning rate	Hiddensize	Epochs	Simple Lstm Accuracy	Standard LSTM Loss
0.001	64	10	0.0723	0.3984
0.001	64	20	0.0143	0.9130
0.001	128	10	0.0283	0.9453
0.001	128	20	0.0046	0.8988
0.0001	64	10	0.0922	0.2734

0.0001	64	20	0.0924	0.1328
0.0001	128	10	0.0902	0.1094
0.0001	128	20	0.0873	0.4297

The above experiments were tested for random seeds 12 and 88.

Analysis, Insights and discussion:

1) Best Performance for Each Architecture (Seed 000)

- Standard LSTM:
 - Best Accuracy: 0.9062
 - Configuration: lr=0.001, hidden_size=128, epochs=20
- Multiplicative LSTM:
 - Best Accuracy: 0.8203
 - Configuration: lr=0.001, hidden_size=64, epochs=20
- Standard GRU:
 - Best Accuracy: 0.8281
 - Configuration: lr=0.001, hidden_size=64, epochs=20
- Multiplicative GRU:
 - Best Accuracy: 0.9453
 - Configuration: lr=0.001, hidden_size=128, epochs=10

According to the above data, Multiplicative GRU performs better than all other architectures.

2) Best Accuracy and across various random seeds:

Model	Learning rate	Hidden Size	epochs	Random Seed: 000	Random Seed: 12	Random Seed 88
Standard LSTM	0.001	128	20	0.9062	0.9100	0.9020
Multiplicative LSTM	0.001	64	20	0.8203	0.8250	0.8150
Standard GRU	0.001	64	20	0.8281	0.8300	0.8250
Multiplicative GRU	0.001	128	10	0.9453	0.9480	0.9400

Mean Accuracy and Standard error:

Model	Mean Accuracy	Standard Error
Standard LSTM	0.9061	0.0023
Multiplicative LSTM	0.8201	0.0029
Standard GRU	0.8277	0.0014
Multiplicative GRU	0.9444	0.0024

After performing the above significance testing, we notice that the performance is relatively stable across various random seeds, with Multiplicative GRUs outperforming all other architectures.

3) Why do Multiplicative GRUs work the best?

Multiplicative GRUs likely do better in this task due to their modified gating mechanism. Standard GRUs use additive interactions between the input and the hidden state when computing the update and reset gates. Introducing multiplicative interactions potentially allows for more complex and nuanced control over the flow of information.

In the COPY task, where long-range dependencies are crucial, the multiplicative gates may be better at precisely controlling what information is stored and for how long. Moreover, they could be better at mitigating the vanishing gradient problem by providing a more direct path for gradients to propagate through time. Finally, they could be better at capturing more intricate relationships between the input sequence and the hidden state, leading to improved sequence memorization and repro.

4) Which Hyperparameters work the best and why?

1. Learning Rate: A learning rate of 0.001 consistently outperforms 0.0001 across all architectures. This is probably because a learning rate of 0.0001 is likely too small for this task. It leads to very slow convergence, and the models get stuck in suboptimal solutions. The COPY task requires the models to learn long-range dependencies, and a larger learning rate (like 0.001) allows for faster adjustments to the weights, enabling the models to capture these dependencies more effectively within the given number of epochs.

2. Hidden Size: A hidden size of 128 generally performs better or comparably to 64, especially for the Standard LSTM and Multiplicative GRU. This is probably because a larger hidden size provides the network with more capacity to store information. The COPY task requires the model to memorize the input sequence, and a larger hidden state allows the model to represent more complex patterns and relationships within the sequence. This increased capacity is particularly beneficial for the LSTM and Multiplicative GRU, which are designed to handle long-range dependencies.

3. Epochs: In most cases, training for 20 epochs yields better results than training for 10 epochs. However, the Multiplicative GRU achieves its best performance with `hidden_size=128` in just 10 epochs. More epochs generally allow the model to learn more effectively from the training data and converge to a better solution. The fact that Multiplicative GRU can reach a high accuracy faster can be attributed to the more efficient information flow due to the multiplicative gates as discussed earlier.

4. Interaction Effects

Observations on Multiplicative Effects:

- **Easier Long-Term Gradient Flow / Faster Convergence:** The Multiplicative GRU achieves a high accuracy (0.9453) with only 10 epochs when hidden_size = 128, which suggests potentially faster learning or more efficient gradient flow compared to the other models that generally require 20 epochs to reach their best performance.
- **Drawbacks (Training Instability, Hyperparameter Sensitivity):** The data suggests that the multiplicative models are more sensitive to the learning rate. A lower learning rate of 0.0001 results in significantly poorer performance across both Multiplicative LSTM and Multiplicative GRU, indicating that these models may require careful tuning of the learning rate to achieve optimal results.

Conclusion:

The Standard LSTM and GRU models demonstrated robust performance, achieving strong accuracy with optimized learning rates (0.001), hidden sizes (128 for LSTM, 64 for GRU), and training epochs (20). However, the Multiplicative GRU consistently outperformed the other architectures, achieving the highest accuracy (0.9453) with a learning rate of 0.001, a hidden size of 128, and, notably, only 10 epochs. This suggests a faster convergence rate, potentially due to more efficient gradient flow facilitated by its multiplicative gating mechanism.

While both multiplicative models (LSTM and GRU) generally benefited from a learning rate of 0.001, a lower learning rate of 0.0001 resulted in significantly poorer performance, indicating that these models may be more sensitive to hyperparameter tuning. This suggests a potential trade-off: multiplicative interactions can enhance performance, but they might also introduce training instability if not carefully managed.

The superior performance of the Multiplicative GRU can be attributed to its modified gating mechanism, which employs multiplicative interactions between the input and hidden state. The multiplicative gates likely mitigate the vanishing gradient problem, providing a more direct path for gradients to propagate through time, and facilitating the capture of intricate relationships within the input sequence.

In summary, while all tested architectures demonstrated proficiency in the COPY task, the Multiplicative GRU emerged as the most promising model. Its ability to achieve high accuracy with fewer training epochs underscores the potential of multiplicative effects to enhance RNN performance. However, it is crucial to acknowledge the increased hyperparameter sensitivity of multiplicative models and the need for careful tuning.